

Javascript

Εισαγωγή

Περιεχόμενα

- Προγραμματισμός από την πλευρά του Πελάτη
- Javascript
- Javascript Document Object Model – D.O.M.
- Javascript – Τρόποι λειτουργίας

Προγραμματισμός από την πλευρά του πελάτη

- Στον προγραμματισμό από την πλευρά του πελάτη, η ιστοσελίδα HTML εμπλουτίζεται με κατάλληλο κώδικα γραμμένο σε κάποια γλώσσα προγραμματισμού και ο οποίος εκτελείται στον υπολογιστή του πελάτη από το πρόγραμμα περιήγησης ιστοσελίδων (Φυλλομετρητής-Web Browser) που χρησιμοποιεί. Ο κώδικας αυτός έχει συνήθως στόχο:
 - Την παροχή δυναμικών εφέ στην ιστοσελίδα.
 - Τον χειρισμό συμβάντων
 - Την αύξηση και τον έλεγχο της διαδραστικότητας μεταξύ του χρήστη και της ιστοσελίδας

Προγραμματισμός από την πλευρά του πελάτη

- Οι γλώσσες προγραμματισμού που χρησιμοποιούνται στον προγραμματισμό από την πλευρά του πελάτη είναι γλώσσες συγγραφής σεναρίων (scripting languages) και με τη βοήθεια εντολών, συναρτήσεων, αντικειμένων και συμβάντων στα αντικείμενα αυτά, παρέχουν:
 - Τη δυνατότητα για τον χειρισμό των αντικειμένων του προγράμματος περιήγησης ιστοσελίδων.
 - Τον χειρισμό των αντικειμένων της ιστοσελίδας (φόρμες, κουμπιά, πίνακες κλπ)
 - Την επεξεργασία των δεδομένων που εμφανίζει η ιστοσελίδα και τη διεξαγωγή υπολογισμών και μετασχηματισμών σε αυτά τα δεδομένα.

Προγραμματισμός από την πλευρά του πελάτη

- Οι πιο γνωστές από αυτές τις γλώσσες είναι η **Javascript** και η **VBScript**. Αυτές οι γλώσσες μαζί με τη χρήση διαδοχικών φύλλων στυλ (**CSS - Cascading Style Sheets**) και HTML μας δίνουν την λεγόμενη **DHTML (Dynamic HTML)**.
- Μάλιστα μετά την **HTML5** και τα **CSS3**, ο συνδυασμός τους με την **Javascript** μας επιτρέπει να κάνουμε σημαντικές εφαρμογές που εκτελούνται από οποιονδήποτε browser σε οποιαδήποτε πλατφόρμα και αυτό αναφέρεται ως **προγραμματισμός σε HTML5**.

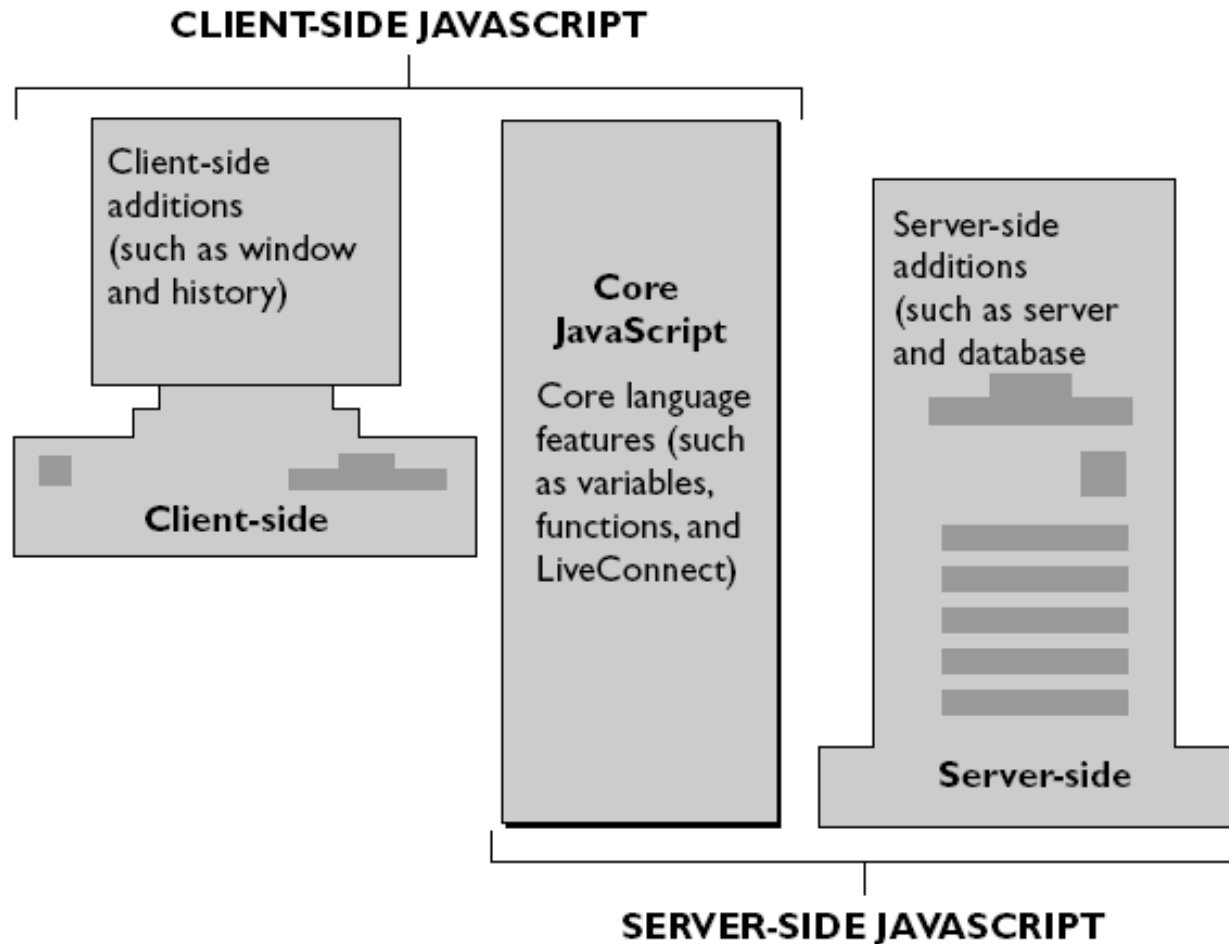
Προγραμματισμός από την πλευρά του πελάτη

- Ο προγραμματισμός από την πλευρά του πελάτη παρουσιάζει μία σειρά περιορισμούς που δεν επιτρέπουν την χρήση του σε εφαρμογές συγκεκριμένων τομέων που απαιτούν πρόσβαση σε συγκεκριμένους πόρους.
- Επίσης επειδή ο κώδικας εκτελείται στον υπολογιστή του πελάτη (από το συγκεκριμένο πρόγραμμα περιήγησης που διαθέτει), περιορίζεται και από ένα σύνολο άλλων παραγόντων όπως το είδος (εταιρεία αλλά και έκδοση) του προγράμματος περιήγησης και των προτιμήσεων που έχει θέσει σε αυτό ο χρήστης (ασφάλεια, λήψη περιεχομένου κλπ).

Javascript

- Η JavaScript είναι μία scripting (δηλαδή εκτελείται με διερμηνευτή) γλώσσα object oriented προγραμματισμού. Ο **πυρήνας** της αποτελείται από ένα βασικό σύνολο αντικειμένων όπως Array, Date, Math κλπ και ένα βασικό σύνολο στοιχείων της γλώσσας (τελεστές, δομές, εκφράσεις, εντολές κλπ). Με αυτόν τον τρόπο η Javascript μπορεί εύκολα να επεκταθεί για διάφορους λόγους προσθέτοντας σε αυτήν διάφορα άλλα αντικείμενα.

Javascript



Javascript

- Έτσι έχουμε:
 - **Client-Side:** Είναι μια επέκταση του πυρήνα της γλώσσας προσθέτοντας αντικείμενα για να ελέγχουμε έναν περιηγητή (browser) ιστοσελίδων και το μοντέλο των αντικειμένων που βρίσκονται σε αυτόν (**Document Object Model, DOM**). Έτσι οι συγκεκριμένες επεκτάσεις δίνουν τη δυνατότητα να αλλάζεις τα στοιχεία του παραθύρου του browser και της ιστοσελίδας (ετικέτες, ιδιότητές τους κλπ), να αντιδράς σε συμβάντα όπως το πάτημα του ποντικιού ή το κλείσιμο ή το άνοιγμα μιας ιστοσελίδας κλπ.

Javascript

- **Server-Side:** Είναι μια επέκταση του πυρήνα της γλώσσας προσθέτοντας αντικείμενα για να χρησιμοποιούμε τη Javascript σε έναν Web Server. Έτσι οι συγκεκριμένες επεκτάσεις δίνουν τη δυνατότητα να έχεις πρόσβαση σε βάσεις δεδομένων, να έχεις πρόσβαση στο σύστημα αρχείων του server κλπ.

Javascript

- Στην παρουσίαση αυτή θα ασχοληθούμε με την client side Javascript.
- Οπότε η client-side Javascript είναι μία scripting γλώσσα που εκτελείται στον browser και με την οποία μπορούμε:
 - να έχουμε πρόσβαση στα αντικείμενα του browser,
 - να διαβάζουμε και να αλλάζουμε τις ιδιότητές των αντικειμένων του και
 - να εκτελούμε κώδικα αντιδρώντας σε συμβάντα που αφορούν ενέργειες του χρήστη, του browser και των αντικειμένων του.

Javascript

- Εδώ θα δούμε κάποια συνοπτικά κάποια βασικά στοιχεία για την γλώσσα javascript.
- Οι εντολές της javascript τελειώνουν με «;»
- Η JavaScript είναι case sensitive οπότε το Alex είναι διαφορετικό από το ALex ή το alex
- Η JavaScript είναι μια γλώσσα με δυναμική γραφή που σημαίνει ότι δεν χρειάζεται να ορίσεις τον τύπο μιας μεταβλητής όταν την δηλώνεις. Θα πάρει τον τύπο της τιμής που θα της αναθέσεις στο πρόγραμμα κάποια στιγμή.

Javascript

- Πχ. Αν δηλώσεις το εξής:
`var answer = 42;`
- και αργότερα βάλεις στην ίδια μεταβλητή ένα string δεν θα δημιουργηθεί κανένα πρόβλημα:
`answer = "Thanks for all the fish...";`
- Σε εκφράσεις string και αριθμούς που ενώνονται με το + η JavaScript τους μετατρέπει σε strings:
`x = "The answer is " + 42; // returns "The answer is 42"`
- Όμως σε εκφράσεις όπως οι επόμενες η JavaScript δεν τους μετατρέπει σε strings:
`"37" - 7; // returns 30`

Javascript

- **Μεταβλητές:** Στην Javascript υποστηρίζονται οι βασικοί τύποι μεταβλητών:
 - Αριθμοί όπως 42, 3.14159.
 - Λογικοί (Boolean), true ή false.
 - Αλφαριθμητικά, όπως “Γεια σας!”.
 - null, μια ειδική λέξη που σημαίνει κενή τιμή (προσοχή η JavaScript είναι case sensitive οπότε το null είναι διαφορετικό από το Null ή το NULL)
 - undefined, μια ιδιότητα που η τιμή της είναι μη ορισμένη

Javascript

- **Μεταβλητές:**
- Στην Javascript μπορούμε να δηλώνουμε μεταβλητές με διάφορους τρόπους:
 1. Βάζοντας απλώς το όνομά τους και την αρχική τους τιμή. Αυτός ο τρόπος πρέπει να αποφεύγεται γιατί τότε οι μεταβλητές γίνονται καθολικές. Π.χ.
`a="nikos"; x=4.5;`

Javascript

2. Βάζοντας `var` και το όνομά τους και αν θέλουμε αρχική τιμή. Τότε γίνονται `local` στο `scope` που δηλώνονται (είτε όλο το πρόγραμμα είτε συνάρτηση). Αν δεν μπει η αρχική τιμή παίρνουν τιμή `undefined` ή `NaN` (Not a Number) αν είναι αριθμός (`NaN` παίρνουν και αν αποθηκευτεί αριθμητική πράξη που δεν μπορεί να δώσει αριθμητικό αποτέλεσμα). Π.χ.

```
var x; var a=23;
```

Javascript

2. Βάζοντας `let` και το όνομά τους και αν θέλουμε αρχική τιμή. Τότε γίνονται `local` στο `scope` που δηλώνονται (είτε όλο το πρόγραμμα, είτε συνάρτηση, είτε `block` μέσα στο οποίο είναι π.χ. μία `if` ή `for`). Π.χ.

```
let a=45.5;
```

2. Πολλές φορές αν ξέρουμε ότι ή τιμή της μεταβλητής δεν θα αλλάξει (άρα θα λειτουργεί ως σταθερά) βάζουμε μπροστά το `const`. Π.χ.

```
const a=45.5;
```

Javascript

- Συναρτήσεις: Για να ορίσουμε μία συνάρτηση γράφουμε:

```
function όνομα(λίστα παραμέτρων) {
```

```
    ...
```

```
}
```

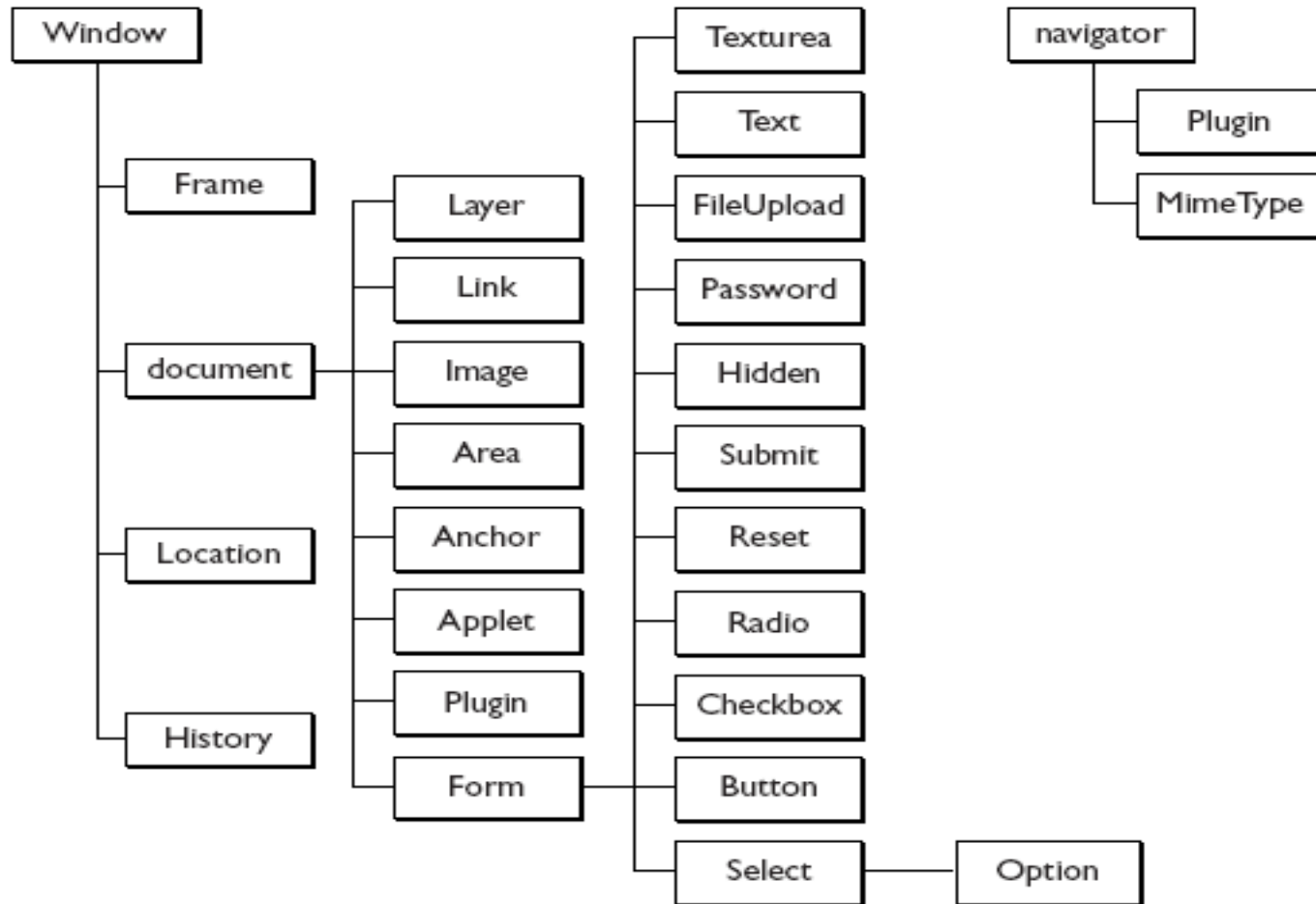
Π.χ.

```
function afun(x, y) {  
    return x+y;  
}
```

Javascript – D.O.M.

- Για να μπορούμε να έχουμε πρόσβαση στα αντικείμενα που αποτελούν έναν browser και την ιστοσελίδα που προβάλλει και να αλληλεπιδρούμε με αυτά έχει αναπτυχθεί το **Document Object Model – D.O.M..**
- Όταν λέμε αντικείμενα του browser εννοούμε το παράθυρο του (window), το έγγραφο HTML (document) καθώς και κάθε ετικέτα που υπάρχει στο έγγραφο.

Javascript – D.O.M.



Javascript – D.O.M.

- Το **Document Object Model** (D.O.M.) είναι η αναπαράσταση δεδομένων των αντικειμένων που αποτελούν τη δομή και το περιεχόμενο ενός εγγράφου στο Web. Μέσω αυτού βλέπουμε πως θα έχουμε πρόσβαση σε κάθε αντικείμενο.
- Χρησιμοποιούμε το D.O.M. ως ένα A.P.I. (Application Programming Interface) για να δημιουργήσουμε web εφαρμογές που εκτελούνται στους browser και να έχουμε πρόσβαση στο περιεχόμενο των εγγράφων αλλά και στον browser. Το D.O.M. αποτελεί μία object-oriented αναπαράσταση των αντικειμένων που μπορούμε να έχουμε πρόσβαση σε αυτά με μία scripting language όπως η JavaScript.

Javascript – D.O.M.

- Μία ιστοσελίδα είναι ένα **document** και το Document Object Model (DOM) αποτελεί μία αναπαράστασή του που μας επιτρέπει να το διαχειριστούμε.
- Όλες οι ιδιότητες, μέθοδοι και events που έχουμε σε μία ιστοσελίδα οργανώνονται ως αντικείμενα (objects). Για παράδειγμα το ίδιο το έγγραφο είναι ένα αντικείμενο (document object), κάθε HTML ετικέτα μέσα στο έγγραφο είναι ένα αντικείμενο κ.ο.κ..

Javascript – D.O.M.

- Για παράδειγμα το DOM καθορίζει ότι το αντικείμενο `document` (που αντιστοιχεί στο HTML έγγραφο) έχει τη μέθοδο `querySelectorAll()` με την οποία μπορούμε να πάρουμε ένα πίνακα με όλα τα αντικείμενα (ετικέτες) ενός τύπου που ανήκουν στο τρέχον HTML έγγραφο. Π.χ.

```
const paragraphs = document.querySelectorAll("p");  
// paragraphs[0] είναι το πρώτο <p> του εγγράφου  
// paragraphs[1] είναι το πρώτο <p> του εγγράφου κλπ.  
alert(paragraphs[0].nodeName);
```

Javascript – D.O.M.

- Το DOM δημιουργείται ως ένα δέντρο αντικειμένων που παριστάνει το έγγραφο. Π.χ. η σελίδα:

```
<html lang="en">
```

```
<head>
```

```
<title>My Document</title>
```

```
</head>
```

```
<body>
```

```
<h1>Header</h1>
```

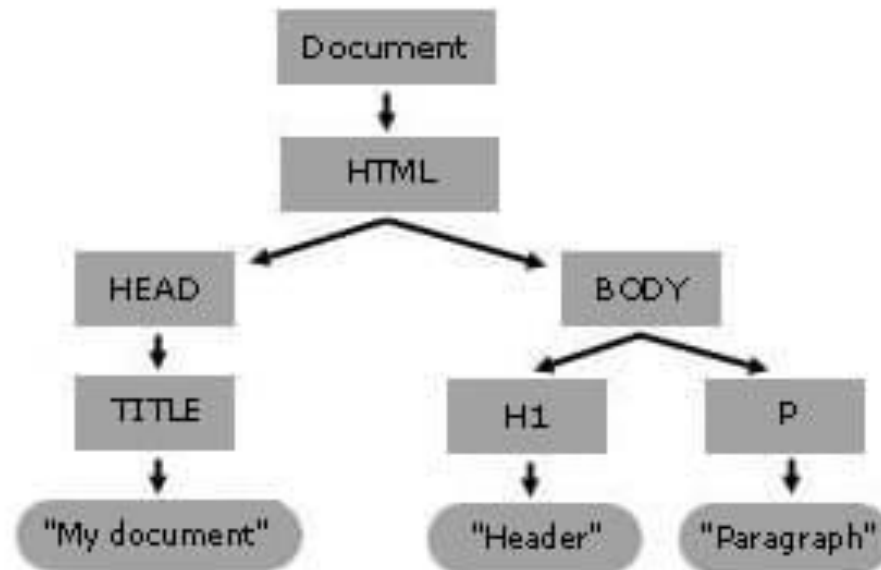
```
<p>Paragraph</p>
```

```
</body>
```

```
</html>
```

Javascript – D.O.M.

- Παριστάνεται ως το παρακάτω δέντρο:



Javascript – D.O.M.

- Όταν εμείς επιθυμούμε να πάρουμε ένα αντικείμενο του εγγράφου για να κάνουμε κάτι χρησιμοποιούμε συνήθως τις παρακάτω μεθόδους:

Javascript – D.O.M.

document.getElementsByTagName("tag name")

Μας επιστρέφει ένα nodeList με όλα τα αντικείμενα αυτού του τύπου «tagname» με τη σειρά που είναι στο έγγραφο

Π.χ.

```
const header =
```

```
document.getElementsByTagName("h2").item(0);
```

//παίρνει το πρώτο h2 του εγγράφου

Javascript – D.O.M.

document.getElementsByName("a name")

Μας επιστρέφει ένα nodeList με όλα τα αντικείμενα που έχουν την ιδιότητά τους name με τιμή «a name» με τη σειρά που είναι στο έγγραφο.

document.getElementById("id value")

Μας επιστρέφει ένα αντικείμενο που έχει την ιδιότητά του id με τιμή «id value».

Javascript – D.O.M.

- Υπάρχουν επίσης και άλλες μέθοδοι όπως:
document.getElementsByClassName("class name")
Μας επιστρέφει ένα nodeList με όλα τα αντικείμενα που έχουν την ιδιότητά τους class με τιμή «class name» με τη σειρά που είναι στο έγγραφο.

π.χ.

document.getElementsByClassName(".pinak1");

Javascript – D.O.M.

`document.querySelector();`

Μας επιστρέφει το πρώτο αντικείμενο που ταιριάζει με το query που έχουμε δώσει παράμετρο

π.χ.

`document.querySelector("p");`

//Μας επιστρέφει την πρώτη παράγραφο

`document.querySelector(".pinak1");`

//Μας επιστρέφει την πρώτη ετικέτα
με class="pinak1 "

Javascript – D.O.M.

document.querySelectorAll();

Μας επιστρέφει ένα NodeList με όλα τα αντικείμενα που ταιριάζουν με το query που έχουμε δώσει παράμετρο, με τη σειρά που είναι στη σελίδα

Javascript – D.O.M.

- Πολλές φορές όταν δουλεύουμε με την Javascript παίρνουμε με διάφορους τρόπους ένα αντικείμενο ή μία συλλογή αντικειμένων και πρέπει να δουλέψουμε με αυτά. Για τον λόγο αυτό καλό είναι να γνωρίσουμε τους παρακάτω τύπους (κλάσεις):

Javascript – D.O.M.

- **NodeList**: Είναι μία συλλογή αντικειμένων που έχουν υπερκλάση το Node. Το Node είναι αφηρημένη κλάση που αποτελεί υπερκλάση κάποιων πολύ χρησιμοποιούμενων κλάσεων όπως Element, Document, DocumentFragment.
- Προσοχή υπάρχουν Live **NodeLists** (δηλαδή αλλαγές στο DOM άμεσα ενημερώνουν τη συλλογή) και Static **NodeLists** (οι αλλαγές στο DOM δεν ενημερώνουν τη συλλογή)

Javascript – D.O.M.

- Π.χ. η ιδιότητα **Node.childNodes** είναι live

```
const parent = document.getElementById("parent");  
let childNodes = parent.childNodes;  
console.log(childNodes.length); // let's assume "2"  
parent.appendChild(document.createElement("div"));  
console.log(childNodes.length); // outputs "3"
```

Javascript – D.O.M.

- Π.χ. η μέθοδος **document.querySelectorAll()** επιστρέφει ένα NodeList που είναι static

Javascript – D.O.M.

- Η NodeList έχει πολλές μεθόδους που μας βοηθάνε να χειριστούμε τα αντικείμενα της συλλογής, όπως:
 - `NodeList.item()`:επιστρέφει ένα αντικείμενο στη συλλογή με βάση τη θέση του (από 0) ή null αν η θέση είναι εκτός ορίων
 - `NodeList.entries ()`:επιστρέφει έναν iterator
 - `NodeList.foreach()`: εκτελεί μία συνάρτηση που δίνουμε παράμετρο για κάθε στοιχείο της συλλογής δίνοντας στη συνάρτηση όρισμα το στοιχείο
 - `NodeList.keys ()`:επιστρέφει έναν iterator για τα κλειδιά
 - `NodeList.values()`:επιστρέφει έναν iterator για τις τιμές

Javascript – D.O.M.

- Ένας συνηθισμένος τρόπος να έχουμε πρόσβαση σε όλα τα αντικείμενα της συλλογής είναι ο:

```
for (let i = 0; i < myNodeList.length; i++) {  
    let item = myNodeList[i];  
}
```

Javascript – D.O.M.

- Ένας άλλος συνηθισμένος τρόπος να έχουμε πρόσβαση σε όλα τα αντικείμενα της συλλογής είναι ο:

```
const list =  
  document.querySelectorAll("input[type=checkbox]");  
for (const checkbox of list) {  
  checkbox.checked = true;  
}
```

Javascript – D.O.M.

- Συχνά χρησιμοποιούμενα Javascript αντικείμενα:
 - **Window:** Γράφοντας window στην javascript αναφερόμαστε στο παράθυρο του browser, να το παράθυρο έχει πλαίσια (frames) υπάρχει και ένα window object για κάθε πλαίσιο. Μέσω του window μπορούμε να έχουμε πρόσβαση σε ιδιότητες και μεθόδους όπως:

Javascript – D.O.M.

- **window.document** που είναι το αντικείμενο που αντιστοιχεί στο έγγραφο HTML που είναι ανοικτό στο παράθυρο και περιέχει όλα τα αντικείμενα του εγγράφου (δηλαδή ένα αντικείμενο για κάθε στοιχείο-ετικέτα του).
- **window.location** που είναι το αντικείμενο που αντιστοιχεί στη διεύθυνση url που έχει φορτωθεί στο παράθυρο.

Javascript – D.O.M.

- **window.console** που είναι το αντικείμενο που αντιστοιχεί στην console του παραθύρου για να μπορούμε να γράφουμε μηνύματα.
- **window.localStorage** και **window.sessionStorage** που είναι τα αντικείμενα για αποθήκευση δεδομένων στον πελάτη(θα τα δούμε παρακάτω)
- κ.λ.π. (δείτε https://www.w3schools.com/jsref/obj_window.asp)

Javascript – D.O.M.

- Συνήθως δεν χρειάζεται να γράφουμε καθόλου το `window` αλλά έχουμε πρόσβαση στα αντικείμενα και τις μεθόδους του απευθείας, π.χ. μπορούμε να γράψουμε `document` και όχι `window.document`

Javascript – D.O.M.

- **document:** Όπως αναφέραμε γράφοντας document (ή window.document) στην javascript αναφερόμαστε στο αντικείμενο που αντιστοιχεί στο έγγραφο HTML που είναι ανοικτό στο παράθυρο και περιέχει όλα τα αντικείμενα του εγγράφου. Μέσω αυτού μπορούμε να πάρουμε όλα τα αντικείμενα του εγγράφου και να διαβάσουμε και να αλλάξουμε τις ιδιότητές τους. Αυτό μπορεί να γίνει με διάφορους τρόπους είτε με τις ιδιότητες είτε με τις μεθόδους που παρέχει. Δείτε στο παρακάτω link για περισσότερες πληροφορίες:

https://www.w3schools.com/jsref/dom_obj_document.asp

Javascript – D.O.M.

Document – Πρόσβαση σε στοιχείο της σελίδας

- Ο πιο συνηθισμένος τρόπος για να πάρουμε ένα αντικείμενο του `document` είναι μέσω της μεθόδου του `getElementById("τιμή της ιδιότητας id")`

Παράδειγμα: Έστω ότι έχουμε το πεδίο:

```
<input type="text" id="on" value="Μπλέ" />
```

Τότε αν στη javascript γράψουμε:

```
var p=document.getElementById("on");
```

Τότε στη μεταβλητή `p` έχουμε το αντικείμενο που αντιστοιχεί στο πεδίο.

Javascript – D.O.M.

- **Ερώτηση:** Πως θα πάρουμε και θα βάλουμε σε μεταβλητές στη javascript τα παρακάτω πεδία:

```
<h1 id="head1"> Μία επικεφαλίδα</h1>
```

```
<input type="number" id="meg" value="70" min="50" max="90" step="5" />
```

```
<input type="reset" id="res" value="Καθαρισμός" />
```

```
<p id="p1">Μία παράγραφος.</p>
```

```
<p id="p2">Άλλη μία παράγραφος.</p>
```

Javascript – D.O.M.

- **Απάντηση:**
- **Για το** `<h1 id="head1">Μία επικεφαλίδα</h1>`
Θα γράψουμε:
`var epik1 = document.getElementById("head1");`
- **Για το** `<input type="number" id="meg" value="70" min=" 50" max=" 90" step="5" />`
Θα γράψουμε:
`var in1 = document.getElementById("meg");`

Javascript – D.O.M.

- Για περισσότερες πληροφορίες δείτε:
 - https://developer.mozilla.org/en-US/docs/Web/API/Document_Object_Model
 - https://www.w3schools.com/js/js_htmlDOM.asp

Javascript – Εισαγωγή στη σελίδα

- Για να εισάγουμε κώδικα javascript σε μία ιστοσελίδα υπάρχουν δύο βασικοί τρόποι:

Javascript – Εισαγωγή στη σελίδα

- Γράφουμε σε όποιο μέρος της σελίδας θέλουμε την ετικέτα `<script>` και ανάμεσα τον κώδικα που θέλουμε. Ο κώδικας (που δεν βρίσκεται σε συνάρτηση) θα εκτελεστεί καθώς φορτώνεται η σελίδα μόλις ο browser φτάσει στο σημείο που βάλαμε το `<script>`. Είναι προτιμότερο να βάζουμε και την ιδιότητα `type` στην ετικέτα, δηλαδή:

```
<script type="text/javascript">
```

```
//κώδικας javascript
```

```
</script>
```

Javascript – Εισαγωγή στη σελίδα

- Γράφουμε τον κώδικα σε ένα ξεχωριστό αρχείο (με κατάληξη .js) και σε όποιο μέρος της σελίδας θέλουμε βάζουμε την ετικέτα `<script>` με την ιδιότητα **src** να έχει τιμή το όνομα του αρχείου (προσοχή να βάλουμε και το κατάλληλο path).

Π.χ.

```
<script src='jsfiles/example.js'></script>
```

Javascript – Εισαγωγή στη σελίδα

- Σε κάθε περίπτωση προτιμάμε την ετικέτα `<script>` και άρα τον κώδικα να τον βάλουμε στο τέλος της σελίδας (πριν το κλείσιμο του `body`) ώστε να εξασφαλίσουμε ότι η σελίδα θα έχει φορτωθεί πριν εκτελεστεί οποιοσδήποτε κώδικας.
- Φυσικά είναι περιπτώσεις που προτιμάμε να τον βάζουμε το `head` ώστε να εκτελεστεί πριν φορτωθεί η σελίδα.

Javascript – Τρόποι Λειτουργίας

- Συνήθως στην javascript λειτουργούμε με δύο τρόπους:

Javascript – Τρόποι Λειτουργίας

- Ο **πρώτος τρόπος** είναι να συνδέουμε ένα αντικείμενο της σελίδας με μία συνάρτηση javascript που θα εκτελεστεί όταν γίνει ένα συμβάν (δες https://www.w3schools.com/js/js_events.asp) σε αυτό το αντικείμενο. Συνήθως αυτό γίνεται σε δύο βήματα
 1. Γράφουμε τη συνάρτηση μέσα στον κώδικα javascript (είτε στην ιστοσελίδα στο script είτε στο εξωτερικό αρχείο)
 2. Συνδέουμε τη συνάρτηση με το αντικείμενο με βάση ένα συμβάν στο αντικείμενο.

Javascript – Τρόποι Λειτουργίας

Παράδειγμα 1 Βήμα 1

Γράφουμε τη συνάρτηση .

Π.χ.

```
function myfun() {  
    alert("πατήθηκε το ποντίκι");  
}
```

Javascript – Τρόποι Λειτουργίας

Συνδέουμε τη συνάρτηση με το αντικείμενο με βάση ένα συμβάν στο αντικείμενο. Αυτό μπορεί να γίνει με δύο τρόπους:

- Μέσα στο HTML αρχείο στην ετικέτα του αντικειμένου με την ιδιότητα που αντιστοιχεί στο συμβάν που θέλουμε. Π.χ.
`<p onclick= "myfun()" > .... </p>`

Javascript – Τρόποι Λειτουργίας

Παράδειγμα 2 Βήμα 1

Π.χ.

`<h1 id="h1" onclick="miasynartisi()"> Μία επικεφαλίδα</h1>`

Και στον κώδικα javascript έχουμε γράψει την συνάρτηση με το ίδιο όνομα:

```
function miasynartisi() {  
    κώδικας συνάρτησης ...  
}
```

- Μόλις κάνουμε click πάνω στο κείμενο της επικεφαλίδας θα κληθεί η συνάρτηση και θα εκτελεστεί ο κώδικά της.

Javascript – Τρόποι Λειτουργίας

- Μέσα στο κώδικα της javascript παίρνουμε το αντικείμενο με μία μέθοδο (συνήθως χρησιμοποιούμε τη μέθοδο `getElementById()` αφού έχουμε βάλει στην ετικέτα του αντικειμένου την ιδιότητα `id`) και καλούμε τη μέθοδο `addEventListener()` του αντικειμένου με πρώτο όρισμα το συμβάν και δεύτερο το όνομα της συνάρτησης). Π.χ.

Javascript – Τρόποι Λειτουργίας

Παράδειγμα 1: Μέσα στο αρχείο HTML

```
<p id="p1"> .... </p>
```

Μέσα στο αρχείο κώδικα javascript (αυτός ο κώδικας πρέπει να μπει εκτός συνάρτησης για να εκτελεστεί ή σε συνάρτηση που κάπως πρέπει να κληθεί ώστε να ενεργοποιηθεί)

```
//Πάρε το αντικείμενο σε μία μεταβλητή
```

```
var elem = document.getElementById("p1");
```

```
//Θέσε να κληθεί η συνάρτηση όταν
```

```
//γίνει το συμβάν click στο αντικείμενο
```

```
elem.addEventListener("click", myfun);
```

Javascript – Τρόποι Λειτουργίας

Παράδειγμα 2: Μέσα στο αρχείο HTML

`<h1 id= "epik1"> Μία επικεφαλίδα</h1>`

και στον κώδικα javascript έχουμε γράψει :

```
var ep1 = document.getElementById("epik1");  
ep1.addEventListener("mouseover", myfun2);  
function myfun2() {  
    κώδικας συνάρτησης ...  
}
```

- Μόλις περάσουμε το δείκτη του ποντικιού πάνω από το κείμενο της επικεφαλίδας θα εκτελεστεί ο κώδικας της συνάρτησης που έχουμε γράψει σαν δεύτερο όρισμα (myfun2()).

Javascript – Τρόποι Λειτουργίας

- Προσοχή στην `addEventListener()` γράφουμε **click** και όχι **onclick** και το όνομα της συνάρτησης δεν είναι σε εισαγωγικά.
- Για περισσότερες πληροφορίες δείτε https://www.w3schools.com/jsref/met_document_addEventListener.asp.

Javascript – Τρόποι Λειτουργίας

- Τα πιο γνωστά συμβάντα είναι:

Event	Description
onchange	An HTML element has been changed
onclick	The user clicks an HTML element
onmouseover	The user moves the mouse over an HTML element
onmouseout	The user moves the mouse away from an HTML element
onkeydown	The user pushes a keyboard key
onload	The browser has finished loading the page

Javascript – Τρόποι Λειτουργίας

- Φυσικά η λίστα με όλα τα συμβάντα είναι πολύ μεγάλη και μπορείτε να τη δείτε στο:
- https://www.w3schools.com/jsref/dom_obj_event.asp

Javascript – Τρόποι Λειτουργίας

- Ο **δεύτερος τρόπος** είναι να εκτελούμε μία συνάρτηση κάθε κάποιο χρονικό διάστημα ώστε να εκτελείται κάθε λίγο. Μέσα σε αυτή τη συνάρτηση βάζουμε τον κώδικα που θέλουμε για να κάνουμε ότι θέλουμε. Ο δεύτερος τρόπος επιλέγεται όταν θέλουμε κάποιος κώδικας να εκτελείται διαρκώς και περιοδικά και όχι ως αποτέλεσμα κάποιου γεγονότος.
- Η διαδικασία αυτή γίνεται για απλούς τρόπους με την μέθοδο `setInterval()` και για πιο προχωρημένες διαδικασίες (π.χ. παιχνίδια) με τη μέθοδο `requestAnimationFrame()`.

Javascript – Τρόποι Λειτουργίας

- Η μέθοδος `setInterval ()` (είναι του αντικειμένου `window` και άρα δεν χρειάζεται να το γράφουμε) παίρνει σαν παραμέτρους τη συνάρτηση και τον αριθμό των `milliseconds` που θα καλείται αυτή η συνάρτηση. Όμως επειδή θα λειτουργεί συνέχεια (μέχρι να κλείσουμε το παράθυρο) αν θέλουμε κάποια στιγμή να το σταματήσουμε πρέπει να καλέσουμε τη μέθοδο `clearInterval()` με παράμετρο το αντικείμενο που μας επέστρεψε η `setInterval()`. Αυτό συνήθως το κάνουμε είτε στην ίδια τη συνάρτηση είτε ως αντίδραση σε ένα συμβάν.

Javascript – Τρόποι Λειτουργίας

- Π.χ.

// κάλεσε την συνάρτηση afun κάθε 2 δευτερόλεπτα

```
var rf = setInterval(afun, 2000);
```

```
function afun()
```

```
{
```

```
    //κώδικας ...
```

```
    if (...) // αν συμβεί κάτι σταμάτα την επανάληψη
```

```
        clearInterval(rf);
```

```
}
```

Javascript – Τρόποι Λειτουργίας

- **Πρόσβαση στις ιδιότητες των αντικειμένων:** Αν πάρω ένα αντικείμενο στην javascript μπορώ να του αλλάξω ότι ιδιότητες έχει μέσω κώδικα βάζοντας το όνομα της ιδιότητας μετά το όνομα χωρισμένα με «.». Παράδειγμα: Αν έχω το `<input type="number" id="meg" value="70" min=" 50" max=" 90" step="5" />`

και έχω στον κώδικα:

```
var in1 = document.getElementById("meg");
```

τότε μπορώ να γράψω:

```
in1.value="85";
```

```
in1.step="10";
```

Javascript – Τρόποι Λειτουργίας

- Μάλιστα μπορώ να γράψω και κατευθείαν (χωρίς να το βάλω σε μεταβλητή):

```
document.getElementById("meg").value="85";
```

```
document.getElementById("meg").step="10";
```

- Επίσης μπορώ να του αλλάξω και τη μορφοποίηση αλλάζοντας όλες τις ιδιότητες CSS που έχει μέσω της ιδιότητάς του style, π.χ.:

```
document.getElementById("meg").style.fontSize="150%";
```

```
document.getElementById("meg").style.color="#ff0000";
```

- Με αυτόν τον τρόπο μπορώ δυναμικά να αλλάξω το έγγραφό παρεμβαίνοντας όπου θέλω όπως και όταν γράφω εξαρχής την σελίδα. Μπορώ ακόμα και να προσθέσω html κώδικα στο έγγραφό μου (ιδιότητα innerHTML του element)!

Canvas

- Στα παραδείγματα αυτά θα δούμε την ετικέτα `canvas` μέσω τις οποίας προσθέτουμε το στοιχείο `canvas`.
- Πρόκειται για νέο στοιχείο της HTML5 με το οποίο (με χρήση javascript) μπορούμε να δημιουργήσουμε εφαρμογές που χρησιμοποιούν και γραφικά και θα εκτελούνται από τον browser σε οποιαδήποτε πλατφόρμα (desktop, mobile κλπ). Με το στοιχείο αυτό δεν έχουμε ανάγκη εξωτερικές εφαρμογές όπως flash, java applets κ.λ.π.

Canvas

- Ο canvas είναι μία περιοχή η οποία χρησιμοποιεί το Canvas 2D Context API (<http://www.w3.org/TR/2dcontext2>) και:
 - μπορούμε να σχεδιάσουμε και μετασχηματίσουμε γραμμές, σχήματα, εικόνες και κείμενο.
 - Μπορούμε να χειριστούμε συμβάντα που προκαλούνται όπως αυτά από τον χρήστη (π.χ. η κίνηση και το πάτημα ποντικιού και πλήκτρων) ή και από το παράθυρο, το έγγραφο κ.λ.π..
- Φυσικά όλα αυτά απαιτούν να χρησιμοποιήσουμε τη γλώσσα προγραμματισμού Javascript.

Canvas

- Για να εισάγουμε έναν canvas στην ιστοσελίδα μας χρησιμοποιούμε την ετικέτα canvas όπως στο επόμενο παράδειγμα:

```
<canvas id="myCanvas" width="400" height="175">
```

Ο Canvas μου !

```
</canvas>
```

- Πρέπει να ορίζουμε την ιδιότητα id γιατί θα τη χρησιμοποιήσουμε για να βρούμε τον canvas στην javascript και να κάνουμε ενέργειες (σχεδίαση κ.λ.π. πάνω σε αυτόν).

Canvas

- Για να κάνουμε χρήση του `canva` θα πρέπει να γράψουμε κώδικα `javascript`. Προτιμάμε να γράφουμε τον κώδικα `javascript` σε ένα ξεχωριστό αρχείο (πάλι με έναν κειμενογράφο όπως ο `notepad++`) το οποίο έχει κατάληξη «.js».

Canvas

- Για να συνδέσουμε το αρχείο javascript με την ιστοσελίδα μας χρησιμοποιούμε την ετικέτα script και την ιδιότητά της src όπως παρακάτω. Εδώ έχουμε συνδέσει με την ιστοσελίδα μας το αρχείο example.js με κώδικα javascript το οποίο βρίσκεται μέσα στον υποφάκελο js:

```
<script src='js/example.js'></script>
```

Canvas

- Η εντολή αυτή είναι καλύτερα να μπαίνει στο τέλος της ιστοσελίδας πριν το κλείσιμο του `<body>` όπως φαίνεται στον επόμενο html κώδικα:

```
<body>
```

```
<canvas id="canvas1" width="400" height="200">
```

```
</canvas><br />
```

```
<script src="js/canvasexample1.js" ></script>
```

```
</body>
```

```
</html>
```

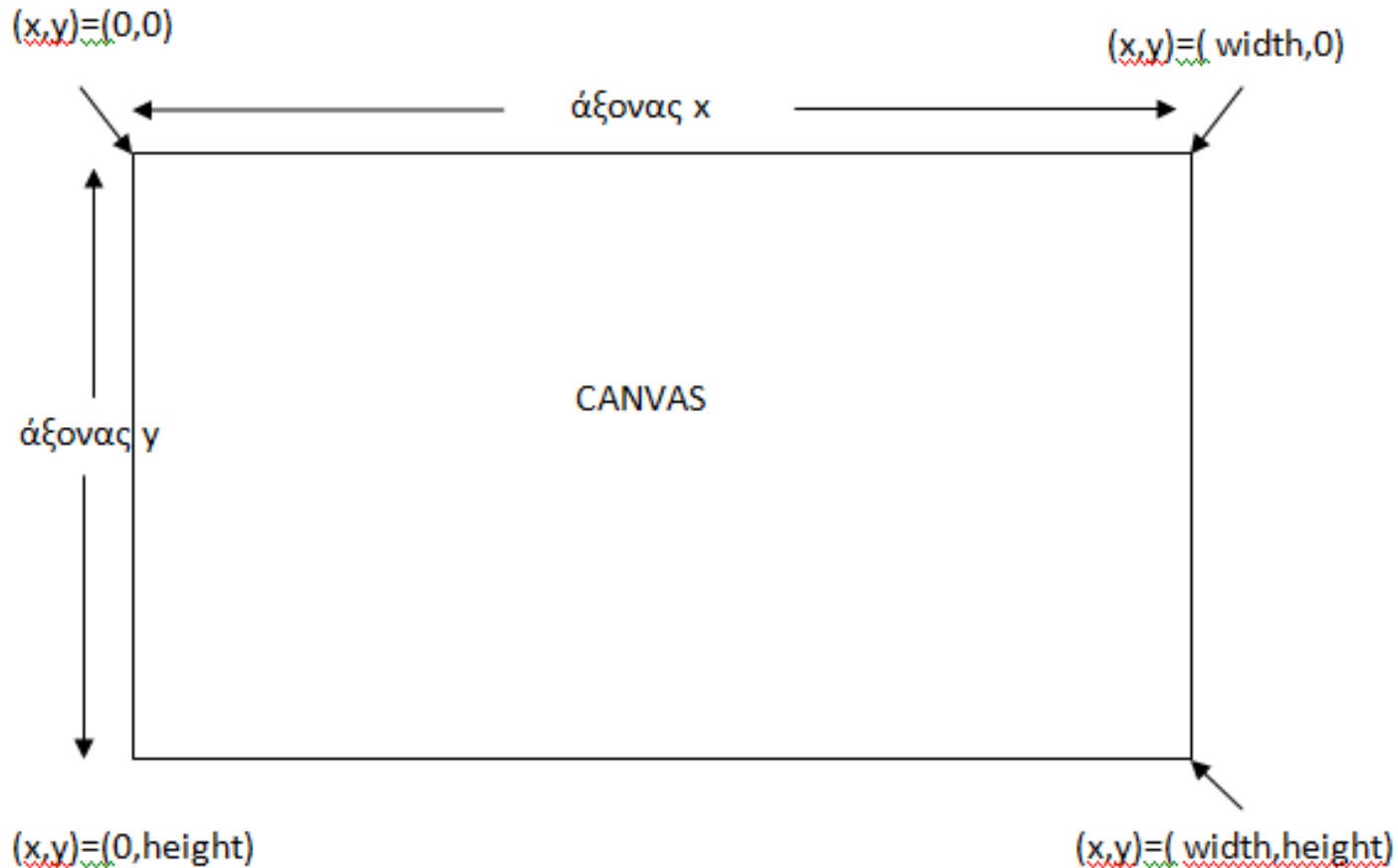
← Συνδέω το αρχείο `canvasexample1.js` που είναι στον υποφάκελο `js` με την ιστοσελίδα μου.

- Για ολοκληρωμένο παράδειγμα δείτε το `canvas_ex1.html` στον φάκελο `example9`.

Canvas

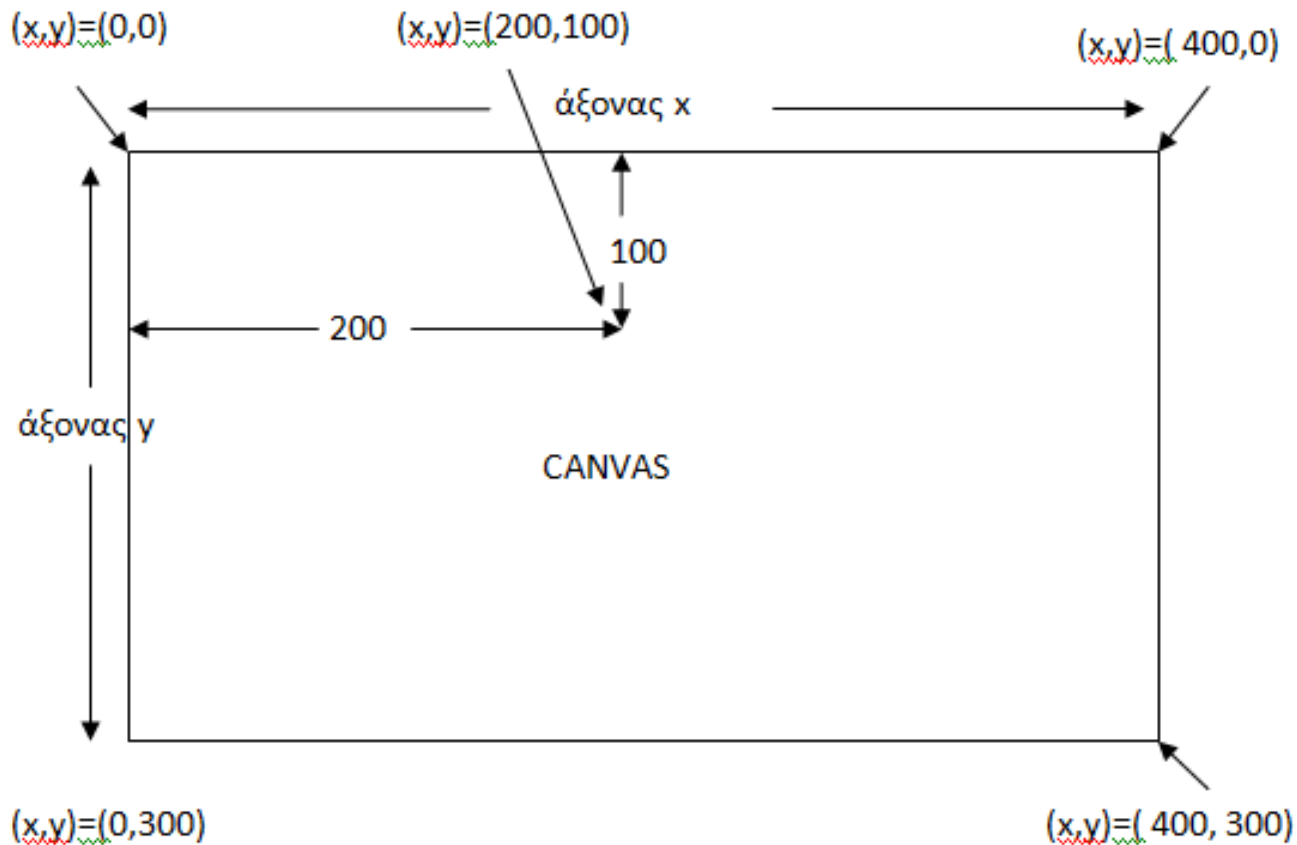
- Πριν ζωγραφίσω στον `canva` πρέπει να κατανοήσουμε δύο βασικά πράγματα:
 - Το πώς είναι το σύστημα συντεταγμένων του `canva` ως περιοχή σχεδίασης. Το χρειάζομαι για να καταλάβω τι τιμές `x`, `y` θα βάζω σαν ορίσματα στις μεθόδους σχεδίασης του `canva` στην `javascript` ώστε να ζωγραφίζω στα σημεία που θέλω.
 - Τη γλώσσα `javascript`.

Canvas – Σύστημα Συντεταγμένων



Canvas – Σύστημα Συντεταγμένων

Canvas 400x300 (πλάτος x ύψος)



Π.Χ.9: Canvas

- Παράδειγμα 1: Δείτε το παράδειγμα 1 που στον φάκελο `example9` και αποτελείται από το αρχείο `canvas_ex1.html` και το αρχείο `canvasexample1.js` που είναι στον υποφάκελο `js`. Δείτε τον τρόπο που συνδέουμε το αρχείο με τον κώδικα `javascript` και μελετήστε τον κώδικα `javascript` με τα σχόλια.
- Η εικόνα κατά την εκτέλεση του παραδείγματος και ο κώδικας HTML φαίνεται στην επόμενη διαφάνεια.

Canvas

Παράδειγμα 1



Καθάρισε τον Canva

```
<canvas id='canvas1'  
        width='400' height='200'>  
Ο Canvas δεν υποστηρίζεται!  
</canvas>  
<br />  
<button onclick="clearCanvas()">  
Καθάρισε τον Canva</button>  
<script src='js/canvasexample1.js'>  
</script>
```

Canvas

Αρχείο canvasexample1.js στον φάκελο javascript:

```
//Πάρε στη μεταβλητή mcanv τον canva που έχει id=canvas1
//Από εδώ και πέρα ο canvas μου είναι στο mcanv
var mcanv = document.getElementById('canvas1');
//Πρόσθεσε ένα event handler για τον canva
//Μόλις συμβαίνει το click θα καλείται η συνάρτηση canvasClicked()
mcanv.addEventListener("click", canvasClicked, false);
//Πάρε το context2d του canva και αποθήκευσέ το στην μεταβλητή ct
//Το context2d περιέχει συναρτήσεις για ζωγραφική κλπ
//Δες https://www.w3.org/TR/2dcontext2
ct = mcanv.getContext('2d');
```

Canvas

//Θέσε στη μεταβλητή ct font, χρώμα γεμίσματος και σχεδίασης

```
ct.font = '38pt Arial';
```

```
ct.fillStyle = 'cornflowerblue';
```

```
ct.strokeStyle = 'blue';
```

//Συνάρτηση που ζωγραφίζει. Ουσιαστικά καθαρίζει ένα

//παραλληλόγραμμο από το 0,0 και με πλάτος, ύψος όσο του canva

```
function drawCanvas(msg, xcoord, ycoord) {
```

```
    //Γέμισε το κείμενο msg ξεκινώντας από το (x,y)=(xcoord,ycoord)
```

```
    ct.fillText(msg, xcoord, ycoord);
```

```
    //Ζωγράφισε το κείμενο msg ξεκινώντας από (x,y)=(xcoord,ycoord)
```

```
    ct.strokeText(msg, xcoord, ycoord );
```

```
}
```

Canvas

// Συνάρτηση που καθαρίζει τον canva

```
function clearCanvas() {  
    // Καθάρισε ένα παραλληλόγραμμο από το σημείο 0,0  
    // και με πλάτος και ύψος όσο του canva  
    ct.clearRect(0, 0, mcanv.width, mcanv.height);  
}
```

// Συνάρτηση που υπολογίζει το σωστό σημείο x και y
// που πατήθηκε (δες box model στα CSS)

```
function windowToCanvas(x, y) {  
    var bbox = mcanv.getBoundingClientRect();  
    return { x: x - bbox.left * (mcanv.width / bbox.width),  
            y: y - bbox.top * (mcanv.height / bbox.height) }; }
```

Canvas

// Συνάρτηση που καλείται όταν το ποντίκι πατιέται μέσα στον canvas

```
function canvasClicked(e) {
```

```
    // Πάρε τις σωστές συντεταγμένες του click
```

```
    var ccoord = windowToCanvas(e.clientX, e.clientY);
```

```
    // Κάλυψε τη συνάρτηση που ζωγραφίζει και δώσε τις τιμές που
```

```
    // φαίνονται σαν τρία ορίσματα
```

```
    drawCanvas('Hello Canvas', ccoord.x, ccoord.y);
```

```
}
```

```
// Κάλυψε τη συνάρτηση που ζωγραφίζει και δώσε της τις τιμές που
```

```
// φαίνονται στα τρία ορίσματα που παίρνει
```

```
drawCanvas('Hello Canvas', mcanv.width/8, (mcanv.height/3)-5);
```

Canvas

- ΑΣΚΗΣΗ 1: Αλλάξτε το παράδειγμα 1 ώστε:
 - Να έχει ο canvas 100px μεγαλύτερο πλάτος.
 - Να χρησιμοποιεί ο canvas άλλο id.
 - Να έχει άλλο χρώμα γεμίσματος και ζωγραφίσματος των γραμμάτων.
 - Όταν πατάμε click μέσα στον canva να εμφανίζεται το μήνυμα «HTML Canvas».
 - Όταν πατάμε click να είναι διαφορετικό το χρώμα γεμίσματος.
 - Όταν πατάμε click να εμφανίζεται το μήνυμα 100px αριστερότερα από εκεί που πατάμε.

Canvas

- Παράδειγμα 2: Δείτε το παράδειγμα 2 που στον φάκελο `example9` και αποτελείται από το αρχείο `canvas_ex2.html` και το αρχείο `canvasexample2.js` που είναι στον υποφάκελο `js`.
- Η εικόνα κατά την εκτέλεση και ο κώδικας HTML φαίνεται στην επόμενη διαφάνεια.

Π.Χ.9: Canvas

Παράδειγμα 2



Γράψε το μήνυμά σου εδώ: HTML5

```
<canvas id='canvas2'  
        width='600' height='250'>
```

Ο Canvas δεν υποστηρίζεται!

```
</canvas>
```

```
<br />
```

```
<label>
```

Γράψε το μήνυμά σου εδώ:

```
<input type="text" id="canvasmsg"  
        value="HTML5" /></label>
```

```
<script src='js/canvasexample2.js'>
```

```
</script>
```

Canvas

- ΑΣΚΗΣΗ 2: Αλλάξτε το παράδειγμα 2 ώστε:
 - Το input πεδίο να έχει άλλο id.
 - Η ταχύτητα επανάληψης να μειωθεί κατά δύο δευτερόλεπτα και ο αριθμός των επαναλήψεων να γίνει 10.
 - Το ύψος, πλάτος και χρώμα του τετραγώνου να αλλάξουν.
 - Όταν πατάμε click αντί να εμφανίζεται παραλληλόγραμμο να εμφανίζεται κύκλος. Δείτε την `arc(x, y, radius, startAngle, endAngle, anticlockwise)` στο https://www.w3schools.com/tags/canvas_arc.asp.

Canvas

- Όταν πατάμε click να εμφανίζεται στο σημείο μία από τις εικόνες που βρίσκονται στον φάκελο `imgs`. Δείτε τις εντολές:

```
var img = new Image();  
img.src = "imgs/chromeimg.jpg";  
context.drawImage(img, x, ccoord.y);
```

- Δείτε μία λύση στο `canvasimg.html`

Canvas

- **Σημείο Ελέγχου:**

1. Γιατί ήταν σημαντική η προσθήκη της ετικέτας `canvas` στην HTML5.
2. Με ποιο τρόπο παίρνουμε από τη javascript τον `canva` που θέλουμε σε μία μεταβλητή;
3. Με ποιο τρόπο παίρνουμε από τη μεταβλητή που έχουμε τον `canva` το `context` για να ζωγραφίζουμε;
4. Πως ορίζουμε το `font` των γραμμάτων, το χρώμα γεμίσματος και ζωγραφίσματος;

Canvas

5. Πως ζωγραφίζουμε (και γέμισμα) ένα παραλληλόγραμμο;
6. Πως ζωγραφίζουμε (και γέμισμα) έναν κύκλο;
7. Πως ζωγραφίζουμε (και γέμισμα) μια γραμμή;
8. Πως ζωγραφίζουμε (και γέμισμα) μια γραμμή;

Αποθήκευση δεδομένων στον πελάτη

- Στα παραδείγματα αυτά θα δούμε τον τρόπο να αποθηκεύουμε δεδομένα στον πελάτη (browser).
- Πρόκειται για τη νέα δυνατότητα Web Storage API που προστέθηκε με την HTML5 (φυσικά με χρήση javascript) και μας δίνει τη δυνατότητα να δημιουργήσουμε πλήρεις εφαρμογές αφού τώρα μπορούμε να αποθηκεύουμε περισσότερα δεδομένα, με μεγαλύτερη ασφάλεια χωρίς να επιβαρύνεται η απόδοση του ιστοτόπου (όπως με τη χρήση cookies).

Αποθήκευση δεδομένων στον πελάτη

- Με το Web Storage API μπορούμε να χρησιμοποιήσουμε στη javascript τα δύο αντικείμενα:
 - `window.localStorage` : Αποθηκεύει δεδομένα χωρίς ημερομηνία λήξης
 - `window.sessionStorage` : Αποθηκεύει δεδομένα μόνο για τη τρέχουσα σύνδεση
- Το window μπορεί να παραλείπεται δηλαδή να γράφουμε σκέτο `localStorage` και `sessionStorage`.

Αποθήκευση δεδομένων στον πελάτη

- Η αποθήκευση δεδομένων γίνεται με ζεύγη όνομα (key) και τιμή (value) και μπορείς να έχεις πρόσβαση στις τιμές με δύο τρόπους:
 - Είτε σαν ιδιότητες των δύο αντικειμένων
 - Είτε με τη χρήση των μεθόδων `setItem(key, value)` και `getItem(key)` των δύο αντικειμένων.

Αποθήκευση δεδομένων στον πελάτη

- Παραδείγματα:

1. Πως θα θέσω τη μεταβλητή `x` ίση με την τιμή 15 για την τρέχουσα σύνοδο;

- Αντιμετωπίζω τη μεταβλητή σαν ιδιότητα του αντικειμένου:

`sessionStorage.x=15;`

- ή καλώ τη μέθοδο `setItem()` του αντικειμένου `sessionStorage` με ορίσματα το όνομα και τη νέα τιμή της μεταβλητής:

`sessionStorage.setItem("x", 15);`

Αποθήκευση δεδομένων στον πελάτη

1. Πως θα πάρω την τιμή της μεταβλητής x από την τρέχουσα σύνοδο και θα την αποθηκεύσω στην μεταβλητή $xval$;
 - Αντιμετωπίζω τη μεταβλητή σαν ιδιότητα του αντικειμένου και έτσι παίρνω την τιμή και την αποθηκεύω όπου θέλω (εδώ σε μια άλλη μεταβλητή):
 $xval = sessionStorage.x$; ή
 - ή καλώ τη μέθοδο `getItem()` του αντικειμένου `sessionStorage` με όρισμα το όνομα και μου επιστρέφεται η τιμή της:
 $xval = sessionStorage.getItem("x");$

Αποθήκευση δεδομένων στον πελάτη

- Πριν προσπαθήσουμε να διαβάσουμε μία τιμή ενός ονόματος θα πρέπει να εξετάζουμε αν υπάρχει, γιατί αν δεν υπάρχει θα συμβεί σφάλμα εκτέλεσης και ο κώδικάς μας θα διακοπεί!
- Για να το ελέγξουμε χρησιμοποιούμε μία if (εντολή ελέγχου) όπως στο επόμενο παράδειγμα:

```
if (localStorage.x) //αν υπάρχει αύξησέ την
    localStorage.x++;
else //αν δεν υπάρχει αρχικοποίησέ την
    localStorage.x = 0;
```

Π.Χ.10: Αποθήκευση δεδομένων στον πελάτη

- Μάλιστα καλό είναι να χρησιμοποιούμε μία συνάρτηση που θα κάνει επιπλέον ελέγχους όπως στο π.χ. στο:

https://developer.mozilla.org/en-US/docs/Web/API/Web_Storage_API/Using_the_Web_Storage_API

Αποθήκευση δεδομένων στον πελάτη

- Παράδειγμα: Δείτε το παράδειγμα `storageex1.html` στον φάκελο `example10`.

Αποθήκευση δεδομένων στον πελάτη

The screenshot shows a web browser window with the address bar displaying `localhost/kekesie/storageex1.html`. The page title is "Παραδείγματα αποθήκευσης". The main content area features a form titled "Φόρμα αποθήκευσης Προτιμήσεων" (Preference Storage Form). This form includes a label "Στοιχεία Εμφάνισης Σελίδας" (Page Display Information), a text input for "Μέγεθος γραμματοσειράς:" (Font size) set to 95, and a color picker for "Χρώμα γραμματοσειράς:" (Font color) set to blue. Below these are buttons for "Θέσε τις επιλογές" (Set options) and "Καθαρισμός" (Reset). The form is followed by two paragraphs of placeholder text in blue: "Μία παράγραφος για να βλέπουμε τα αποτελέσματα." and "Άλλη μία παράγραφος για να βλέπουμε τα αποτελέσματα."

The bottom of the image shows the browser's developer tools, specifically the "Αποθήκευση" (Storage) tab. It displays a table of stored data for the current page:

Κλειδί	Τιμή
fcol	#0000ff
fsize	95

On the left side of the developer tools, a list of storage items is shown, including "Cookies", "Αποθήκευση συνεδρίας" (Session Storage), "Προσωρινή αποθήκευση" (Local Storage), and "Τοπική αποθήκευση" (IndexedDB), all for the domain `http://localhost`.

Αποθήκευση δεδομένων στον πελάτη

The screenshot shows a web browser window with the address bar displaying `localhost/kekesie/storageex1.html`. The page title is "Παραδείγματα αποθήκευσης". The main heading is "Φόρμα αποθήκευσης Προτιμήσεων" in red. Below it is a form with the following elements:

- Label: "Στοιχεία Εμφάνισης Σελίδας"
- Field: "Μέγεθος γραμματοσειράς:" with a value of 110 and a percentage sign.
- Field: "Χρώμα γραμματοσειράς:" with a red color picker.
- Buttons: "Θέσε τις επιλογές" and "Καθαρισμός"

Below the form, there are two lines of red text:

- Μία παράγραφος για να βλέπουμε τα αποτελέσματα.
- Άλλη μία παράγραφος για να βλέπουμε τα αποτελέσματα.

The bottom of the browser shows the developer console with the "Κονσόλα" tab selected. It displays four log entries:

Message	Source
Θέτω νέες τιμές 95, #ffff00	storageex1.html:55:17
Θέτω νέες τιμές 95, #ff0000	storageex1.html:55:17
Θέτω νέες τιμές 95, #0000ff	storageex1.html:55:17
Θέτω νέες τιμές 110, #ff0000	storageex1.html:55:17

Αποθήκευση δεδομένων στον πελάτη

- ΑΣΚΗΣΗ 1: Αλλάξτε το παράδειγμα `storageex1.html` ώστε:
 - Να έχει άλλο ένα πεδίο για το χρώμα φόντου της σελίδας.
 - Να έχει άλλο ένα πεδίο για το μέγεθος γραμμάτων που θα αλλάζει μόνο το μέγεθος γραμμάτων της παραγράφου με `id="p1"` που υπάρχει μετά τη φόρμα.
 - Να αποθηκεύει τις αλλαγές για την παράγραφο με `id="p1"` στη `sessionStorage` και όχι στη `localStorage`. Μετά δοκιμάστε να κλείσετε το παράθυρο και δείτε τις διαφορές.
 - Δημιουργείστε ένα φάκελο `js` και μέσα σε αυτόν ένα αρχείο `stcode.js` και μεταφέρετε όλον τον javascript κώδικα που είναι στο τέλος του αρχείου `storageex1.html` στο αρχείο αυτό. Μετά βάλτε στο τέλος του `storageex1.html` τη σωστή δήλωση για να συνδεθεί με τη σελίδα ο κώδικας.

Αποθήκευση δεδομένων στον πελάτη

- ΑΣΚΗΣΗ 2: Δοκιμάστε να φτιάξετε μια απλή εφαρμογή calculator που:
 - έχει δύο πεδία στα οποία θα γράφουμε 2 αριθμούς και ένα πεδίο στο οποίο θα φαίνεται το αποτέλεσμα.
 - έχει δύο κουμπιά για πρόσθεση και αφαίρεση των αριθμών.
 - Έχει δύο κουμπιά, ένα για αποθήκευση του αποτελέσματος στη μνήμη και ένα για φόρτωση του αποτελέσματος από τη μνήμη στο πρώτο πεδίο.

Αποθήκευση δεδομένων στον πελάτη

Αναφορές:

- https://www.w3schools.com/html/html5_webstorage.asp
- https://developer.mozilla.org/en-US/docs/Web/API/Web_Storage_API/Using_the_Web_Storage_API

Αναφορές

- W3C (<https://www.w3.org/>)
- MDN Web Docs Mozilla (<https://developer.mozilla.org/en-US/docs/Learn>)
- W3Schools (<https://www.w3schools.com/>)
- Wikipedia (<https://en.wikipedia.org>)
- Σημειώσεις προγράμματος